

Lecture 10: IEEE Floating-Point Numbers and Machine Precision

Math 6610

C. Mou

Department of Mathematics and Statistics
Utah State University

Fall 2026

Today's Goals

- Write a real number in normalized binary form
- Read the IEEE single and double layouts
- Define machine precision $\epsilon_{\text{mach}} = 2^{-52}$ and the rounding rules
- Bound the representation error: $|\text{fl}(x) - x| / |x| \leq \frac{1}{2} \epsilon_{\text{mach}}$

Representing Real Numbers

Problem: how do we store $x \in \mathbb{R}$ on a computer?

Storage is finite, so $\sqrt{3} = 1.73205\dots$, and even large integers, cannot be stored exactly.

Pick a base $\beta \geq 2$: $\beta = 2$ binary, $\beta = 10$ decimal. For an integer $x \geq 0$,

$$\begin{aligned}x &= b_n \beta^n + b_{n-1} \beta^{n-1} + \dots + b_1 \beta + b_0 \\ &=: (b_n b_{n-1} \dots b_1 b_0)_\beta, \quad b_i \text{ integers, } 0 \leq b_i \leq \beta - 1.\end{aligned}$$

Example (53 in base 10 and base 2).

$$\begin{aligned}53 &= 5 \cdot 10^1 + 3 \cdot 10^0 = (53)_{10} \\ &= 32 + 16 + 4 + 1 = 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 \\ &= (110101)_2 \Rightarrow 6 \text{ bits.}\end{aligned}$$

Why Floating Point, Not Fixed Point

Example (*How many numbers fit in 52 bits?*). With $x = (b_{51}b_{50} \cdots b_1b_0)_2$ we get $x = 0, 1, 2, \dots, 2^{52} - 1$: about $4.5 \cdot 10^{15}$ values, spaced 1 apart.

That spacing is *absolute*, and scientific computing needs 10^{-300} and 10^{300} in the same program.

Remark (*The floating-point idea*). Fix the number of significant digits and let an exponent place the binary point. The representable numbers then have constant *relative* spacing $\approx 2^{-52}$, which is how error actually propagates through multiplication and division. It is also why conditioning and stability are stated in relative errors.

The Normalized Binary Form

Base $\beta = 2$. Every $x \in \mathbb{R}$ with $x \neq 0$ can be written as

$$x = \pm(1.b_1b_2b_3\cdots) \times 2^p, \quad b_1, b_2, \cdots \in \{0, 1\}, \quad p \in \mathbb{Z}.$$

Remark. The leading digit is forced to be 1: that is what *normalized* means. Since it is always 1, it need not be stored, which buys one free bit of precision.

Remark (Two pieces to store). The bits $b_1, b_2, \dots$ (the *mantissa*) and the integer p (the *exponent*). A computer stores finitely many of each.

Converting a Decimal by Hand: 41.7

Integer part: divide by 2, read the remainders upward. Fractional part: multiply by 2, read the integer parts downward.

$41/2 = 20$	r 1	$2 \times 0.7 = 1.4$	bit 1, keep 0.4
$20/2 = 10$	r 0	$2 \times 0.4 = 0.8$	bit 0, keep 0.8
$10/2 = 5$	r 0	$2 \times 0.8 = 1.6$	bit 1, keep 0.6
$5/2 = 2$	r 1	$2 \times 0.6 = 1.2$	bit 1, keep 0.2
$2/2 = 1$	r 0	$2 \times 0.2 = 0.4$	bit 0, keep 0.4 (repeats)
$1/2 = 0$	r 1		

The fractional part cycles, so the bits repeat:

$$\begin{aligned} 41.7 &= (101001.1\overline{0110})_2 \\ &= +(1.010011\overline{0110}) \times 2^5. \end{aligned}$$

Why the Two Schemes Work

Integer part: repeated division

If $41 = b_5 2^5 + \cdots + b_1 2 + b_0$, dividing by 2 leaves remainder b_0 and quotient $b_5 2^4 + \cdots + b_1$. Each division peels off the next bit, least significant first.

Fractional part: repeated multiplication

If $0.7 = b_{-1} 2^{-1} + b_{-2} 2^{-2} + \cdots$, then $2 \times 0.7 = 1.4$ has integer part $b_{-1} = 1$ and fractional part $0.4 = b_{-2} 2^{-1} + \cdots$. Each doubling exposes the next bit, most significant first.

Remark. The fractional parts of 0.7 cycle: $.7 \rightarrow .4 \rightarrow .8 \rightarrow .6 \rightarrow .2 \rightarrow .4$, so the bits repeat forever. Most finite decimals are infinite binaries. Rounding is unavoidable, even for input as ordinary as 41.7 or 0.1.

IEEE Layouts

$$\underbrace{\pm}_{\text{sign}} \left(1. \underbrace{b_1 b_2 \dots b_N}_{\text{mantissa}} \right) \times 2^{\underbrace{p}_{\text{exponent}}}, \quad p \text{ stored in } M \text{ bits.}$$

Precision	sign	N (mantissa)	M (exponent)	total bits	bytes
Single	1	23	8	32	4
Double	1	52	11	64	8
Long Double	1	64	15	80	10

Remark. From now on: double precision. The sign, mantissa and exponent together fill $1 + 52 + 11 = 64$ bits, and the hidden leading 1 is free.

Double Precision

A double is

$$\pm(1.b_1 b_2 \dots b_{52}) \times 2^p, \quad p \text{ an integer in } [-1022, 1023].$$

Two numbers to keep in mind:

$$\begin{aligned} 1 &= +1.\underbrace{00 \dots 0}_{52} \times 2^0, \\ 1 + 2^{-52} &= +1.\underbrace{00 \dots 0}_{51} 1 \times 2^0. \end{aligned}$$

Remark. Nothing lies between them: $1 + 2^{-52}$ is the smallest double larger than 1. The gap 2^{-52} is the spacing of the grid near 1.

Machine Precision

Loop over $n = 0, 1, 2, \dots$ and test

$$1 + 2^{-n} \stackrel{?}{>} 1.$$

The test fails first at $n = 53$: the stored result of $1 + 2^{-53}$ is exactly 1.

Definition (*Machine precision*).

$$\epsilon_{\text{mach}} := 2^{-52} = (1 + 2^{-52}) - 1 = 2.220 \dots \times 10^{-16}$$

(eps in MATLAB). This is why double precision carries about 16 significant decimal digits.

Remark. ϵ_{mach} measures *relative* spacing, not absolute. Near 2^p the gap between neighbouring doubles is $2^p \epsilon_{\text{mach}}$.

Rounding: the Map fl

$$x \in \mathbb{R} \xrightarrow{\text{round to nearest}} \text{fl}(x) = \text{the double closest to } x.$$

Write $x = \pm 1.b_1b_2 \cdots b_{52} b_{53} \cdots \times 2^p$. The bits $b_{53}, b_{54}, \dots$ must go.

Remark (*Four rules*).

- *Chopping* (= round down): drop $b_{53}, b_{54}, \dots$
- *Round up*: add 1 to b_{52} .
- *Round to nearest*: down if $b_{53} = 0$; up if $b_{53} = 1$ and some later bit is 1.
- *Exact tie* ($b_{53} = 1$, all later bits 0): round so that b_{52} ends at 0 – “ties to even”.

Remark. Ties-to-even is the IEEE default. Always rounding up would bias a long computation in one direction.

The Relative Error of $\text{fl}(x)$

Definition (Representation error). For $x \neq 0$,

$$\left| \frac{\text{fl}(x) - x}{x} \right| \leq \frac{1}{2} \epsilon_{\text{mach}} = 2^{-53} = 1.11 \dots \times 10^{-16}.$$

Equivalently $\text{fl}(x) = x(1 + \epsilon)$ with $|\epsilon| \leq \frac{1}{2} \epsilon_{\text{mach}}$.

Why

Let $x = (1.b_1b_2 \dots) \times 2^p$, so $2^p \leq |x| < 2^{p+1}$. Doubles with exponent p sit on a grid of spacing $2^p \cdot 2^{-52}$, and rounding to nearest lands within half a step:

$$|\text{fl}(x) - x| \leq \frac{1}{2} \cdot 2^{p-52} = 2^{p-53}, \quad \left| \frac{\text{fl}(x) - x}{x} \right| \leq \frac{2^{p-53}}{2^p} = 2^{-53}.$$

The form $\text{fl}(x) = x(1 + \epsilon)$ is what we will use in every stability proof.

Rounding 41.7

From the conversion, the mantissa bits are 010011 followed by 0110 repeating:

$$41.7 = +1.\underbrace{010011}_6 \underbrace{0110\ 0110\ \dots\ 0110}_{44 = 11 \times 4} \underbrace{01}_{b_{51}b_{52}} \mid \underbrace{10}_{b_{53}b_{54}}\ 0110 \dots \times 2^5$$

Here $b_{53} = 1$ and later bits are not all 0, so we round up: add 1 to b_{52} , turning the final 01 into 10.

$$\text{fl}(41.7) = +1.010011 \underbrace{0110 \dots 0110}_{44} 10 \times 2^5$$

Remark. Check in MATLAB: `num2hex(41.7)` gives 4044d9999999999a; the final hex digit $a = 1010_2$ shows the rounded-up ending.

Numbers Worth Remembering

$\epsilon_{\text{mach}} = 2^{-52}$	$\approx 2.22 \times 10^{-16}$
$\frac{1}{2}\epsilon_{\text{mach}} = 2^{-53}$ (rounding bound)	$\approx 1.11 \times 10^{-16}$
significant decimal digits (double)	$\approx 15\text{--}16$
largest double	$\approx 1.80 \times 10^{308}$
smallest normalized double 2^{-1022}	$\approx 2.23 \times 10^{-308}$

In MATLAB: `eps`, `realmax`, `realmin`. Then try `0.1 + 0.2 == 0.3`, which is false: each of 0.1, 0.2, 0.3 is an infinite binary, rounded separately.

Looking Ahead

- **Next lecture:** Floating-Point Arithmetic and Round-Off Errors